

GITHUB full

Complete GitHub Workflow Guide: Single Person & Multi-Person Collaboration

This guide provides accurate, step-by-step commands and best practices for both **Single-Person (Solo)** development and **Multi-Person Collaboration** (covering both **Shared Repository** and **Fork & Pull Request** workflows).

1. Single Person (Solo) Workflow

When working alone on a project across one or multiple computers (e.g., Desktop and Laptop).

A. Initial Setup (Once per project)

```
# Option A: Clone an existing GitHub repository
git clone https://github.com/YourUsername/your-repo.git
cd your-repo

# Option B: Turn a local folder into a GitHub repository
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YourUsername/your-repo.git
git push -u origin main
```

B. Daily Development Loop

```
# 1. ALWAYS pull before starting work (essential if using multiple devices or web
edits)
git pull origin main

# 2. Make your edits / add files...

# 3. Check what files changed
git status

# 4. Stage and commit your changes
git add .
git commit -m "Add dynamic programming knapsack implementation"

# 5. Push to GitHub
git push origin main
```

C. Solo Feature Branching (Recommended for major changes)

```
# Create and switch to a test/feature branch
git checkout -b refactor-code

# Edit, commit as usual
git add .
git commit -m "Refactor utility functions"

# Merge back to main when ready
git checkout main
git merge refactor-code
git push origin main

# Delete local feature branch
git branch -d refactor-code
```

2. Multi-Person Collaboration Workflows

There are two common ways to collaborate on GitHub. Choose the model that fits your team:

MODEL 1: Shared Repository Workflow (RECOMMENDED for Team & Group Projects)

Best for: Course projects, hackathons, and small teams where everyone has direct push permissions to the main repo.

One-Time Setup (Owner & Collaborators)

1. **Owner:** On GitHub, go to **Settings** → **Collaborators** → **Add people** → Invite team members.
2. **Collaborators:** Accept email/GitHub invite.
3. **Collaborators:** Clone the repository directly:

```
git clone https://github.com/OwnerUsername/project-repo.git
cd project-repo
```

Daily Team Workflow (Branch & PR)

```
# Step 1: Ensure main is up-to-date
git checkout main
git pull origin main

# Step 2: Create a feature branch for your specific task
git checkout -b feature/huffman-coding
```

```
# Step 3: Work, stage, and commit
git add .
git commit -m "Implement Huffman encoding tree construction"

# Step 4: Push feature branch to shared GitHub repository
git push -u origin feature/huffman-coding

# Step 5: Open a Pull Request (PR) on GitHub
# Go to GitHub repo page -> Click 'Compare & pull request' -> Submit PR.

# Step 6: Code Review & Merge
# Teammates review code on GitHub -> Click 'Merge pull request'.

# Step 7: Local Sync & Clean Up (All team members run this)
git checkout main
git pull origin main
git branch -d feature/huffman-coding # Delete local branch
git push origin --delete feature/huffman-coding # (Optional) Delete remote branch
```

MODEL 2: Fork & Pull Request Workflow

Best for: Open-source contributions or projects where collaborators do NOT have direct write access to the central repository.

One-Time Setup for Collaborators

1. **Fork on GitHub:** Click **Fork** on <https://github.com/OwnerUsername/project-repo>.
2. **Clone & Link Upstream:**

```
# Clone collaborator's own fork
git clone https://github.com/CollaboratorUsername/project-repo.git
cd project-repo

# Add central repo as 'upstream'
git remote add upstream https://github.com/OwnerUsername/project-repo.git

# Verify remotes (should list origin [fork] and upstream [owner])
git remote -v
```

Daily Collaborator Workflow

```
# Step 1: Sync main branch with central owner repo
git checkout main
git pull upstream main

# Step 2: Create feature branch
git checkout -b feature/karatsuma-mul
```

```
# Step 3: Make edits, stage, commit
git add .
git commit -m "Add Karatsuba multiplication logic"

# Step 4: Push feature branch to THEIR fork ('origin')
git push -u origin feature/karatsuba-mul

# Step 5: Submit Pull Request on GitHub
# Go to Collaborator Fork -> Click 'Compare & pull request' -> Select base: Owner/main.

# Step 6: Owner Merges PR on GitHub

# Step 7: Sync Fork after Merge & Clean Up
git checkout main
git pull upstream main
git push origin main # Keep fork's main branch in sync
git branch -d feature/karatsuba-mul # Delete used feature branch
```

3. Workflow Comparison Summary

Scenario	Workflow Type	Remote Names	Key Pull Command
Solo Developer	Direct Main or Local Branch	origin	git pull origin main
Team / Group Project	Shared Repo Branching	origin	git pull origin main
Open Source / Guest Contributor	Fork & Pull Request	origin (Fork), upstream (Owner)	git pull upstream main

4. Common Edge Cases & Troubleshooting

A. Resolving Merge Conflicts

If Git reports a conflict during `git pull` or `git merge`:

```
# 1. Check conflicting files
git status

# 2. Open conflicting files in editor, look for conflict markers:
# <<<<<<< HEAD
# Your local code
# =====
# Incoming code
# >>>>>>> branch-name

# 3. Edit the file to keep desired code, remove markers, then:
```

```
git add .  
git commit -m "Fix merge conflict"
```

B. Saving Uncommitted Work before Branch Switching (`git stash`)

```
# Save uncommitted changes temporarily  
git stash  
  
# Switch branches or pull updates  
git checkout main  
git pull origin main  
  
# Restore stashed changes  
git stash pop
```

C. Ignoring Files (`.gitignore`)

Always create a `.gitignore` file at the root of your project to prevent pushing temporary files or binaries:

```
# Python  
__pycache__/  
*.pyc  
  
# C/C++ compiled binaries  
*.exe  
*.out  
*.o  
  
# Environment / OS files  
.DS_Store  
.vscode/  
.idea/
```